

[Database](#) > ... > [Prisma](#) > [Connecting with Prisma](#) >

# Prisma

This quickly shows how to connect your Prisma application to Supabase Postgres. If you

Database

- If you plan to solely use Prisma instead of the Supabase Data API (PostgREST), turn it off in the [API Settings](#).

## 1 Create a custom user for Prisma

In the [SQL Editor](#), create a Prisma DB user with full privileges on the public schema.

This gives you better control over Prisma's access and makes it easier to monitor using Supabase tools like the [Query Performance Dashboard](#) and [Log Explorer](#).

- password manager

For security, consider using a [password generator](#) for the Prisma role.

```
1  -- Create custom user
2  create user "prisma" with password 'custom_password' bypassrls createdb;
3
4  -- extend prisma's privileges to postgres (necessary to view changes in Dashboard)
5  grant "prisma" to "postgres";
6
7  -- Grant it necessary permissions over the relevant schemas (public)
8  grant usage on schema public to prisma;
```

```
9 grant create on schema public to prisma;
10 grant all on all tables in schema public to prisma;
11 grant all on all routines in schema public to prisma;
12 grant all on all sequences in schema public to prisma;
13 alter default privileges for role postgres in schema public grant all on tables to prisma;
14 alter default privileges for role postgres in schema public grant all on routines to prisma;
15 alter default privileges for role postgres in schema public grant all on sequences to prisma;
```

```
1 -- alter prisma password if needed
2 alter user "prisma" with password 'new_password';
```



## 2 Create a Prisma Project

Create a new Prisma Project on your computer

Create a new directory

```
1 mkdir hello-prisma
2 cd hello-prisma
```

Initiate a new Prisma project

npm   pnpm   yarn   bun

```
1 npm init -y
2 npm install prisma typescript ts-node @types/node --save-dev
3
4 npx tsc --init
5
6 npx prisma init
```

### 3 Add your connection information to your .env file

On your project dashboard, click [Connect](#)

Find your Supavisor Session pooler string. It should end with 5432. It will be used in your `.env` file.

If you're in an [IPv6 environment](#) or have the IPv4 Add-On, you can use the direct connection string instead of Supavisor in Session mode.

If you plan on deploying Prisma to a serverless or auto-scaling environment, you'll also need your Supavisor transaction mode string.

The string is identical to the session mode string but uses port 6543 at the end.

server-based deployments

serverless deployments

In your `.env` file, set the `DATABASE_URL` variable to your connection string

```
1 # Used for Prisma Migrations and within your application
2 DATABASE_URL="postgres://[DB-USER].[PROJECT-REF]:[PRISMA-PASSWORD]@[DB-REGION].pooler
```

Change your string's `[DB-USER]` to `prisma` and add the password you created in step 1

```
1 postgres://prisma.[PROJECT-REF]...
```

### 4 Create your migrations

If you have already modified your Supabase database, synchronize it with your migration file. Otherwise create new tables for your database

New Projects

Populated Projects

## Create new tables in your prisma.schema file

```
1  model Post {
2    id      Int      @id @default(autoincrement())
3    title   String
4    content String?
5    published Boolean @default(false)
6    author  User?    @relation(fields: [authorId], references: [id])
7    authorId Int?
8  }
9
10 model User {
11   id      Int      @id @default(autoincrement())
12   email   String   @unique
13   name    String?
14   posts   Post[]
15 }
```

## commit your migration

npm   pnpm   yarn   bun

```
1  npx prisma migrate dev --name first_prisma_migration
```

### 5 Install the prisma client

Install the Prisma client and generate its model

npm   pnpm   yarn   bun

```
1  npm install @prisma/client
2  npx prisma generate
```

6

## Test your API

Create a `index.ts` file and run it to test your connection

```
1  const { PrismaClient } = require('@prisma/client');
2
3  const prisma = new PrismaClient();
4
5  async function main() {
6    //change to reference a table in your schema
7    const val = await prisma.<SOME_TABLE_NAME>.findMany({
8      take: 10,
9    });
10   console.log(val);
11 }
12
13 main()
14   .then(async () => {
15     await prisma.$disconnect();
16   })
17   .catch(async (e) => {
18     console.error(e);
19     await prisma.$disconnect();
20     process.exit(1);
21   });
```

Edit this page on GitHub [↗](#)

🔗 Need some help? [Contact support](#)

🔔 Latest product updates? [See Changelog](#)

✅ Something's not right? [Check system status](#)

—

Contributing

Author Styleguide

Open Source

SupaSquad

Privacy Settings

